VIT Bhopal University

Department of Computer Science & Engineering

Academic Session: 2025 – 2026

PROJECT REPORT

Food Calorie Estimator

Using Machine Learning Model

Linear Regression · Graph Traversal (BFS / DFS) · Health Analytics

Submitted By

Student Name	Nehal Sonal
Registration Number	25BHI10068
Course	B.Tech CSE (Health Informatics)
Institution	VIT Bhopal University
Academic Session	2025 – 2026
Guided By	Prof. [Faculty Name]

Abstract

This is a simple application with great utility, developed with the aim of showing how Machine Learning and the Graph Traversal Algorithm can be combined to solve a particular problem in real life. The application is quite simple, where the user will be prompted to input either the name of the food or the values of the nutrients found in the food, such as protein, carbs, fats, etc., and the application will return the result with the values of the food's calories, nutrients, etc.

The application will utilize the “Linear Regression” Algorithm, which is considered to be the easiest to understand and implement within the Machine Learning scope, to predict the values of the food's calories. In addition, the application will create a graph where all the food items will be interconnected according to their categories. Then, two algorithms for graph traversal, namely BFS and DFS, will be applied to the graph to assist the user in finding the food items according to their categories.

This application will be included within a single Python program.

1. Introduction

The ability to monitor what we consume is more important now more than ever. This is because the aim of an individual could be to lose weight, gain more muscles, or eat healthy food. However, in order to do any of these, the individual must know the number of calories and nutrients contained in a particular food.

This project is unique in the sense that we didn’t use a pre-existing tool but created our own based on the ideas we have learned in class thus far.

From one perspective, Linear Regression helps us find the mathematical relationship between the nutrients and the number of calories contained in food using the data given. From the other perspective, we utilized a graph data structure that shows the relationship between food and other food items using BFS and DFS algorithms. Both perspectives are intended to satisfy the curiosity of an individual regarding what they are consuming.

2. Problem Statement

The majority of people are not aware of the calories and nutrients contained in the food they consume on a daily basis. They may search for this data online but cannot find similar foods or other foods that belong to the same food group.

The problems addressed in this project:

•How do we search for the calories, protein, carbohydrates, and fats contained in a specific food item?

•How do we calculate the calories contained in a food item if we know the macronutrients contained in it?

•How do we search for other foods similar to or belonging to the same food group as a specific food item?

These three problems can be addressed using this program, which incorporates data sets, ML, and graph algorithms like BFS and DFS.

3.Objectives

•Prepare a dataset of 32 foods and their calorie and macronutrient content.

•Implement a Linear Regression model to predict calories from protein, carbohydrates, and fat content.

•Create a food graph where foods are nodes and foods within the same category are connected.

•Use BFS to traverse all foods similar to a specific food item, i.e., foods within the same category.

•Use DFS to traverse all foods within a specific category of a food item.

•Present all above features through a simple CLI interface in the form of numbered options, which can be used by anyone.

4. Methodology

(A) Dataset :

Instead of an external CSV file, all food data is stored directly inside the Python program as a list of tuples named fooddata. Each tuple in this list contains six elements: food name, calorie count, protein in grams, carbohydrates in grams, fat in grams, and food category.

The data set includes 32 food items divided into six categories: Protein, Carbs, Fruit, Vegetable, Dairy, and Fats. These food categories include all the common food groups that an individual would encounter in their day-to-day life. The nutritional values provided per 100g serving are standard.

```
fooddata = [
    # (food data, calories, protein in g, carbs in g, fat in g, category)
    ("Chicken Breast",  165, 31.0,  0.0,  3.6, "Protein"),
    ("Egg",              78,  6.0,  0.6,  5.0, "Protein"),
    ("Tuna",            132, 28.0,  0.0,  1.0, "Protein"),
    ("Paneer",          296, 18.0,  3.4, 22.0, "Protein"),
    ("Lentils",         116,  9.0, 20.0,  0.4, "Protein"),
    ("Chickpeas",       164,  9.0, 27.0,  2.6, "Protein"),
    ("Tofu",             76,  8.0,  2.0,  4.2, "Protein"),
    ("White Rice",      206,  4.0, 45.0,  0.4, "Carbs"),
    ("Brown Rice",      215,  5.0, 45.0,  1.6, "Carbs"),
    ("Oats",            154,  5.0, 28.0,  2.5, "Carbs"),
    ("Wheat Bread",      79,  3.0, 15.0,  1.0, "Carbs"),
    (" Potato",     90,  2.0, 21.0,  0.1, "Carbs"),
    ("Pasta",           220,  8.0, 43.0,  1.3, "Carbs"),
    ("Corn",            132,  5.0, 29.0,  1.8, "Carbs"),
    ("Banana",           89,  1.1, 23.0,  0.3, "Fruit"),
    ("Apple",            52,  0.3, 14.0,  0.2, "Fruit"),
    ("Mango",            60,  0.8, 15.0,  0.4, "Fruit"),
    ("Orange",           47,  0.9, 12.0,  0.1, "Fruit"),
    ("Grapes",           67,  0.6, 17.0,  0.4, "Fruit"),
    ("Watermelon",       30,  0.6,  8.0,  0.2, "Fruit"),
    ("Broccoli",         55,  3.7, 11.0,  0.6, "Vegetable"),
    ("Spinach",          23,  2.9,  3.6,  0.4, "Vegetable"),
    ("Carrot",           41,  0.9, 10.0,  0.2, "Vegetable"),
    ("Tomato",           18,  0.9,  3.9,  0.2, "Vegetable"),
    ("Cucumber",         16,  0.7,  3.6,  0.1, "Vegetable"),
    ("Milk",             42,  3.4,  5.0,  1.0, "Dairy"),
    ("Yogurt",           59, 10.0,  3.6,  0.4, "Dairy"),
    ("Cheese",          402, 25.0,  1.3, 33.0, "Dairy"),
    ("Butter",          717,  0.9,  0.1, 81.0, "Fats"),
    ("Almonds",         579, 21.0, 22.0, 50.0, "Fats"),
    ("Peanut Butter",   588, 25.0, 20.0, 50.0, "Fats"),
    ("Olive Oil",       884,  0.0,  0.0,100.0, "Fats"),
]
```

(B) Machine Learning Model

Once the Data Frame is created, the ML model is trained immediately upon starting the program. We use a linear regression algorithm. The three columns for macronutrients (proetinin_g, carbin_g, fatin_g) will be used as predictors, and the calories column will be the target.

The data will be divided 80/20 for training and testing using train_test_split. Upon training, it will immediately learn how to relate nutrient composition to calorie count, or essentially how many calories come from each gram of each nutrient. This happens silently in the background, and the user will never see any output.

```
X = data[["proetinin_g", "carbin_g", "fatin_g"]]#training of Machine model
y = data["calories"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
```

When the user selects Option 2, the prediction function is invoked. This function accepts protein, carbs, and fat as arguments, creates a pandas DataFrame containing the arguments and the correct column names to prevent warnings from the library, and finally makes a rounded prediction.

```
def prediction(protein, carbs, fat):
    input = input = pd.DataFrame([[protein, carbs, fat]], columns=["proetinin_g", "carbin_g", "fatin_g"])
    predicted = model.predict(input)[0]
    return round(predicted, 2)
```

(C) Food Graph Construction :

Once the model is trained, the program constructs a graph. The graph is represented as an adjacency list in the form of a Python dictionary called 'graph.' Each food item is a key in the dictionary, and the list of food items it is connected to is the value.

Two food items are connected by an edge if and only if they belong to the same category. So, for example, Apple, Banana, Mango, Orange, Grapes, and Watermelon are all in the Fruit category and are connected to each other. This is a total of six clusters, each representing a category.

```
graph = {}# Food Graph is being built here - each food is a node, edges connect foods in the same category
for food in data["foodi"]:
    graph[food] = []

for i, row_i in data.iterrows():
    for j, row_j in data.iterrows():
        if i != j and row_i["category"] == row_j["category"]:
            a = row_i["foodi"]
            b = row_j["foodi"]
            if b not in graph[a]:
                graph[a].append(b)
            if a not in graph[b]:
                graph[b].append(a)
```

(D) BFS Implementation

BFS stands for Breadth First Search. It is a graph search algorithm. BFS visits all nodes at a given breadth before moving to nodes at the next breadth. In other words, it visits all neighbors of a given node before visiting their neighbors, and so on.

BFS is being utilized in Option 3 of this program to collect all foods similar to the one entered by the user. This is because all foods within a certain category are neighbors to each other. The main data structure utilized here is a deque, or double-ended queue, from Python's collections module.

```
def bfs(start): # Using BFS we Find similar foods level by level
    visited = []
    queue = deque()
    queue.append(start)
    visited.append(start)
    while queue:
        current = queue.popleft()
        for neighbor in graph[current]:
            if neighbor not in visited:
                visited.append(neighbor)
                queue.append(neighbor)
    visited.remove(start)
    return visited
```

(E) DFS Implementation

DFS stands for Depth First Search. Depth First Search is a graph traversal algorithm that explores as far as it can in one direction before backtracking and going in another direction. Depth First Search is a recursive algorithm in this project.

In Option 4, DFS is used to explore the category of a particular food in its entirety. This is because it calls itself on its unvisited neighbors until it has visited all nodes connected to it. This gives us a complete list of foods in this category in depth-first order.

```
def dfs(start, visited=None):# exploring the category of a food recursively through DFS
    if visited is None:
        visited = []
    visited.append(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(neighbor, visited)
    return visited
```

5. Tools and Technologies Used

Various tools and libraries are utilized in this project to assist in the development and implementation of the project.

Python, version 3.13.x, is utilized as the main programming language in developing the application and implementing all the functionalities of the project.

Pandas, the latest version, is utilized to load and process data sets in an efficient manner, enabling the processing of structured data in the form of data frames.

NumPy, the latest version, is utilized to prepare and process numerical data sets in the form of arrays, which are critical in machine learning predictions and computations.

Scikit-learn, the latest version, is utilized to build the machine learning model, including functionalities to perform linear regression and split the data set into training and test data sets.

Visual Studio Code, the latest version, is utilized as the code editor to write and manage the code of the project.

6. Program Structure and Menu

The application provides various options that enable users to perform different operations on the food data.

1. The first option allows users to perform a search on a particular food item. The user enters the food item of their choice, and the system provides information regarding the food's nutritional content, including calories, protein, carbohydrates, fat, and category.
2. The second option allows users to predict the calories of a food item using the nutrients provided. The feature uses machine learning to predict the calories of a food item based on the nutrients provided by the user.
3. The third option allows users to perform a search on foods of the same category using Breadth-First Search (BFS) traversal. The system uses BFS to traverse the tree and list foods of the same category as the food item provided by the user.
4. The fourth option allows users to perform a Depth-First Search (DFS) traversal to list all food items of the same category.
5. The fifth option gives a complete view of the food data set, showing all 32 food items in a structured and formatted manner.
6. The sixth option gives the user an option to exit the program, terminating it and displaying a goodbye message.

7. Results

A. Option 1 — Food Search

When the user types a food name, the program looks it up in the DataFrame using a case-insensitive match and displays all its nutritional details. For example, searching for Apple gives:

```
----- FOOD CALORIE ESTIMATOR USING ML MODEL -----

 MENU OPTIONS:
1. Search the food you want to eat
2. Calorie prediction from input nutrients
3. Find similar foods using BFS
4. Exploring food category using DFS
5. View complete food dataset
6. Exit
************************************************************
Enter your choice (1-6): 1

Enter food name: Apple

 Results for: Apple
  Calories      : 52 kcal
  Protein       : 0.3 g
  Carbohydrates: 14.0 g
  Fat           : 0.2 g
  Category      : Fruit
```

B. Option 2 — Calorie Prediction

The user enters macronutrient values and the trained ML model predicts the calorie content. This is useful for foods not present in the dataset:

```
 MENU OPTIONS:
1. Search the food you want to eat
2. Calorie prediction from input nutrients
3. Find similar foods using BFS
4. Exploring food category using DFS
5. View complete food dataset
6. Exit
************************************************************
Enter your choice (1-6): 2

Enter the nutritional values of your food:
  Protein (g)       : 56
  Carbohydrates (g): 77
  Fat (g)           : 89

 Nutritional Summary
  Protein       : 56.0 g
  Carbohydrates: 77.0 g
  Fat           : 89.0 g
  Predicted Calories: 1348.85 kcal
```

C. Option 3 — BFS Similar Foods

BFS starts from the entered food and collects all neighbors (same category foods) level by level:

```
 MENU OPTIONS:
1. Search the food you want to eat
2. Calorie prediction from input nutrients
3. Find similar foods using BFS
4. Exploring food category using DFS
5. View complete food dataset
6. Exit
**********************************************************
Enter your choice (1-6): 3

Enter food name to find similar categories: Banana

 Foods similar to 'Banana':
  1. Apple
  2. Mango
  3. Orange
  4. Grapes
  5. Watermelon
```

D. Option 4 — DFS Category Exploration

DFS recursively explores the full category of the entered food :

```
 MENU OPTIONS:
1. Search the food you want to eat
2. Calorie prediction from input nutrients
3. Find similar foods using BFS
4. Exploring food category using DFS
5. View complete food dataset
6. Exit
**********************************************************
Enter your choice (1-6): 4

Enter food name to explore its category: Banana

 Full category explored from 'Banana':
  1. Apple
  2. Mango
  3. Orange
  4. Grapes
  5. Watermelon
```

8. Conclusion

This project ended up being a good learning experience. Going into it, I wasn't really sure how Machine Learning and Graph Traversal would mesh, but once I started working on it, it made sense. The Machine Learning side deals with the numeric side, predicting calories from nutrient information, while the graph side deals with the relational side, understanding how foods relate to each other

The Linear Regression model is appropriate for this dataset since the relationship between the macronutrients and the calories is a linear relationship. The BFS and DFS codes, although simple, are doing exactly what they should be doing. It provides a level-by-level structure of similar foods with BFS and a comprehensive depth-first search of a particular category with DFS.

Writing all of the code in one file, without any external dependencies, has made the project more convenient to run and share. It has also forced me to think more critically about how everything connects to everything else. The dataset connects to both the ML model and the graph builder, and the graph connects to both BFS and DFS.

Overall, this mini project has done exactly what it set out to do, proving that two very different aspects of computer science can be combined in one working application.

.

9. Future Enhancements

- Add hundreds of food items to the dataset for more accurate results for the ML model.
- Add a GUI using tkinter or a web interface using Flask.
- Add a feature where users can add their own food items to the dataset.
- Add a feature where users can track how many calories they have consumed in a day using a meal log.
- Consider using weighted edges for the graph based on the similarity of the nutrition content between two food items, rather than the category.
- Try using more complex models, such as Random Forest or Gradient Boosting, for more accurate results.
- Add a feature where users are given a meal plan for the day based on the number of calories they want to consume.

10. References

- USDA FoodData Central — https://fdc.nal.usda.gov
- scikit-learn Documentation — https://scikit-learn.org/stable/
- Python Official Documentation — https://docs.python.org/3/
- GeeksforGeeks — BFS and DFS Algorithms — https://www.geeksforgeeks.org
- Cormen, T.H. et al. — Introduction to Algorithms (Graph Traversal chapters)
- Kaggle — Beginner Machine Learning Tutorials — https://www.kaggle.com/learn